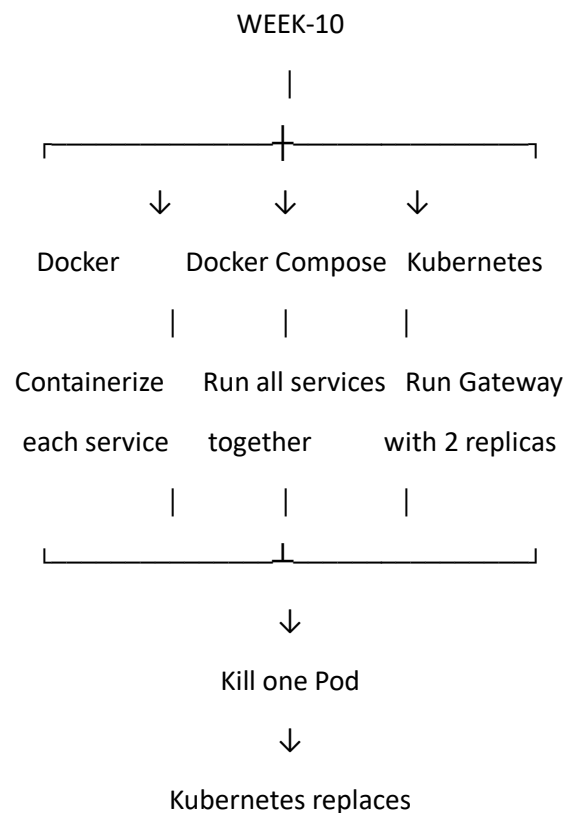


PRACTICAL WEEK-10 Containerized Hosting & Orchestration

[Vikram Immella, 2520090118]



1. Project Structure and Environment Verification

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> docker --version
Docker version 29.8.1, build 4a63305
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> docker compose ver
sion
Docker Compose version v5.5.1
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> docker info
Client:
Version:      29.8.1
Context:      desktop-linux
Debug Mode:   false
Plugins:
agent: Docker AI Agent Runner (Docker Inc.)
  Version:    v1.141.0
  Path:       C:\Users\DELL\.docker\cli-plugins\docker-agent.exe
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
  Version:    v1.31.0
  Path:       C:\Users\DELL\.docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
  Version:    v0.37.1
  Path:       C:\Users\DELL\.docker\cli-plugins\docker-buildx.ex
```

2. Docker Compose Configuration Validation

```

PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow_Week10_Fresh_Implementation\bookflow-week10> docker compose con
fig
name: bookflow-week10
services:
  db-mongo:
    container_name: bookflow-mongo
    image: mongo:7
    networks:
      default: null
    ports:
      - mode: ingress
        target: 27017
        published: "27017"
        protocol: tcp
    volumes:
      - type: volume
        source: mongo-data
        target: /data/db
        volume: {}
  db-postgres:
    container_name: bookflow-postgres
    environment:
      POSTGRES_DB: bookflow
      POSTGRES_PASSWORD: postgres
      POSTGRES_USER: postgres
    image: postgres:16
    networks:
      default: null
    ports:
      - mode: ingress

```

3. Docker Images Built Successfully for All BookFlow Services

```

[+] build 3/3
✓Image bookflow-gateway:week10      Built      100.0s
✓Image bookflow-user-service:week10  Built      99.8s
✓Image bookflow-stock-service:week10 Built      100.1s
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow_Week10_Fresh_Implementation\bookflow-week10> |

```

4. BookFlow Multi-Container Stack Running Successfully

```

PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow_Week10_Fresh_Implementation\bookflow-week10> docker compose up -d
[+] 35/5535
✓Image mongo:7          Pulled      42.8s
✓Image postgres:16      Pulled      37.3s
✓Volume bookflow-week10_postgres-data Created     0.1s
✓Network bookflow-week10_default Created     0.1s
✓Volume bookflow-week10_mongo-data Created     0.8s
✓Container bookflow-postgres Started      1.6s
✓Container bookflow-mongo Started      2.8s
✓Container bookflow-user-service Started      1.8s
✓Container bookflow-stock-service Started      2.7s
✓Container bookflow-gateway Started      2.1s
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow_Week10_Fresh_Implementation\bookflow-week10> docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED     STATUS      PORTS
bookflow-gateway    bookflow-gateway:week10             "uvicorn app:app --h..."            gateway    24 seconds ago Up 21 seconds 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
bookflow-mongo      mongo:7                              "docker-entrypoint.s..."            db-mongo   26 seconds ago Up 23 seconds 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
bookflow-postgres   postgres:16                         "docker-entrypoint.s..."            db-postgres 26 seconds ago Up 22 seconds 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
bookflow-stock-service bookflow-stock-service:week10       "java -jar app.jar"                  stock-service 24 seconds ago Up 22 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
bookflow-user-service bookflow-user-service:week10        "docker-entrypoint.s..."            user-service 24 seconds ago Up 23 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp

```

5A. FastAPI Gateway

IMMELLA VIKRAM's Workspace

Event & Room Occupancy Management System > Health > Health Check

GET http://localhost:8000/health

Body

```

1 {
2   "service": "bookflow-gateway",
3   "week": "10",
4   "author": "Immella Vikram",
5   "status": "UP"
6 }

```

6. FastAPI Gateway Successfully Communicates with Node.js User Service

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow_Week10_Fresh_Implementation\bookflow-week10> curl.exe http://localhost:8080/gateway/users [{"userId":501,"name":"Kamtam Priyanka","role":"Student"}]
```

7. Spring Boot Stock Service — Inventory Retrieved from PostgreSQL

```
_Week10_Fresh_Implementation\bookflow-week10> curl.exe -i "http://localhost:8080/inventory/101"
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked
Date: Mon, 05 Oct 2026 05:23:34 GMT

{"bookId":101,"totalStock":5,"availableStock":5}
```

8. Spring Boot Stock Service — Inventory Retrieved from PostgreSQL

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow_Week10_Fresh_Implementation\bookflow-week10> docker exec -it bookflow-postgres psql -U postgres -d bookflow -c "SELECT * FROM inventory ORDER BY book_id;"
 book_id | available_stock | total_stock
-----+-----+-----
      101 |                5 |           5
      102 |                3 |           3
      103 |                0 |           2
(3 rows)
```

9. Spring Boot Stock Service — Successful Book Borrowing

IMMELLA VIKRAM's Workspace

Event & Room Occupancy Management System > Health > Heal

POST http://localhost:8080/orders

Body

```
1 {
2   "userId": 501,
3   "bookId": 101
4 }
```

Body

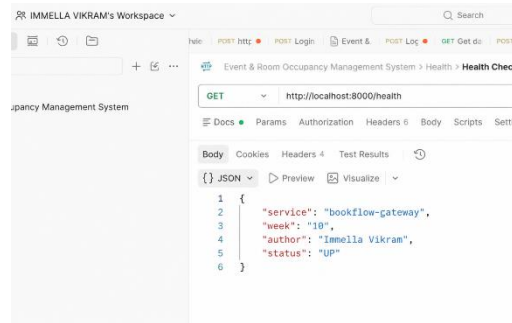
```
1 {
2   "userId": 501,
3   "status": "BORROWED",
4   "id": 1,
5   "bookId": 101
6 }
```

10. PostgreSQL — Inventory Updated After Successful Book Borrowing Stock

availability is changed from 5→4 for book_id = 101

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> docker exec -it bo
okflow-postgres psql -U postgres -d bookflow -c "SELECT * FROM i
nventory ORDER BY book_id;"
 book_id | available_stock | total_stock
-----+-----+-----
    101 |          4       |           5
    102 |          3       |           3
    103 |          0       |           2
```

11. FastAPI Gateway — Order Successfully Forwarded to Spring Boot



12. PostgreSQL — Gateway Order and Inventory Records Verified

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> docker exec -it bo
okflow-postgres psql -U postgres -d bookflow -c "SELECT * FROM i
nventory ORDER BY book_id;"
 book_id | available_stock | total_stock
-----+-----+-----
    101 |          3       |           5
    102 |          3       |           3
    103 |          0       |           2
```

13. Docker Desktop Kubernetes Cluster — Node Ready

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> kubectl get nodes
NAME                STATUS    ROLES    AGE    VERSION
desktop-control-plane Ready    control-plane 79s    v1.36.1
```

14. Kubernetes Deployment — BookFlow Gateway Running with 2 Replicas

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> kubectl apply -f .
\k8s\deployment.yaml
deployment.apps/bookflow-gateway created
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> kubectl get deploy
ments
NAME                READY    UP-TO-DATE    AVAILABLE    AGE
bookflow-gateway    0/2      2             0            6s
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow
_Week10_Fresh_Implementation\bookflow-week10> kubectl get pods -
o wide
NAME                READY    STATUS    RESTARTS
AGE    IP            NODE                NOMINATED NODE    READ
INESS GATES
bookflow-gateway-dcf45cb66-cvzpl 1/1      Running    0
12s    10.244.0.5    desktop-control-plane <none>             <non
e>
bookflow-gateway-dcf45cb66-wcwbq 1/1      Running    0
12s    10.244.0.6    desktop-control-plane <none>             <non
e>
```

15. Kubernetes LoadBalancer Service and Pod Endpoints

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow_Week10_Fresh_Implementation\bookflow-week10> kubectl get service
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
bookflow-gateway-service LoadBalancer 10.96.245.127 <pending>      8000:31755/TCP 9s
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP    10m
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\BookFlow_Week10_Fresh_Implementation\bookflow-week10> kubectl get endpoints b
ookflow-gateway-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                ENDPOINTS    AGE
bookflow-gateway-service 10.244.0.5:8000,10.244.0.6:8000 4m11s
```

16. Kubernetes Service — FastAPI Gateway Health Check Successful

```
PS C:\WINDOWS\system32> curl.exe -i "http://localhost:8000/health"
HTTP/1.1 200 OK
date: Mon, 05 Oct 2026 06:11:15 GMT
server: uvicorn
content-length: 83
content-type: application/json

{"service":"bookflow-gateway","week":"10","author":"Kamtam Priyanka","status":"UP"}
```

17. Kubernetes Deployment Before Self-Healing Test

```
PS C:\WINDOWS\system32> kubectl get pods
NAME                                READY   STATUS    RESTARTS
AGE
bookflow-gateway-dcf45cb66-cvzpl    1/1     Running   0
21m
bookflow-gateway-dcf45cb66-wcwbq    1/1     Running   0
21m
PS C:\WINDOWS\system32> kubectl get deployment bookflow-gateway
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
bookflow-gateway    2/2     2             2           21m
```

18. Kubernetes Self-Healing — Failed Pod Automatically Replaced

```
PS C:\WINDOWS\system32> kubectl delete pod bookflow-gateway-dcf45cb66-cvzpl
pod "bookflow-gateway-dcf45cb66-cvzpl" deleted from default namespace
PS C:\WINDOWS\system32> kubectl get pods -w
NAME                                READY   STATUS    RESTARTS
AGE
bookflow-gateway-dcf45cb66-bkrp9    1/1     Running   0
11s
bookflow-gateway-dcf45cb66-wcwbq    1/1     Running   0
75m
bookflow-gateway-dcf45cb66-wcwbq    1/1     Running   0
76m
```

19. Kubernetes Deployment Restored to 2/2 Replicas

```
PS C:\WINDOWS\system32> kubectl get deployment bookflow-gateway
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
bookflow-gateway    2/2     2             2           77m
```

Folder Structure:

```
BookFlow_Week10_Fresh_Implementation/
├── bookflow-week10/
│   ├── docker-compose.yml
│   ├── README.md
│   └── gateway/
```

```

|   ├── Dockerfile
|   ├── requirements.txt
|   └── app.py
|
|   ├── user-service/
|   |   ├── Dockerfile
|   |   ├── package.json
|   |   └── server.js
|   |
|   ├── stock-service/
|   |   ├── Dockerfile
|   |   ├── pom.xml
|   |   └── src/
|   |       ├── main/
|   |       |   ├── java/
|   |       |   |   ├── com/bookflow/week10/
|   |       |   |   |   ├── StockServiceApplication.java
|   |       |   |   |   ├── controller/
|   |       |   |   |   |   ├── StockController.java
|   |       |   |   |   |   ├── entity/
|   |       |   |   |   |   |   ├── Inventory.java
|   |       |   |   |   |   |   ├── LibraryOrder.java
|   |       |   |   |   |   |   └── repository/
|   |       |   |   |   |   |       ├── InventoryRepository.java
|   |       |   |   |   |   |       └── LibraryOrderRepository.java
|   |       |   |   └── resources/
|   |       |   |       └── application.yml
|   |       └── k8s/
|   |           ├── deployment.yaml
|   |           └── service.yaml

```

Codes:

□ gateway/Dockerfile

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

```

□ gateway/app.py

```
import os import httpx from fastapi import FastAPI,
HTTPException app = FastAPI(title="BookFlow Gateway",
version="Week-10", description="Containerized gateway by
Kamtam Priyanka")
USER_SERVICE_URL = os.getenv("USER_SERVICE_URL", "http://localhost:3000")
STOCK_SERVICE_URL = os.getenv("STOCK_SERVICE_URL", "http://localhost:8080")
@app.get("/health") async
def health():
    return {"service":"bookflow-gateway","week":"10","author":"Kamtam
Priyanka","status":"UP"}
@app.get("/gateway/users") async def
get_users():    async with
httpx.AsyncClient() as client:
    r = await client.get(f"{USER_SERVICE_URL}/users")    if
r.status_code >= 400: raise HTTPException(status_code=r.status_code,
detail=r.text)    return r.json() @app.post("/gateway/orders") async def
create_order(order: dict):
    async with httpx.AsyncClient() as client:
        r = await client.post(f"{STOCK_SERVICE_URL}/orders", json=order)
    if r.status_code >= 400: raise HTTPException(status_code=r.status_code,
detail=r.text)
    return r.json()
```

- gateway/requirements.txt

```
fastapi==0.115.6
uvicorn[standard]==0.34.0 httpx==0.28.1
```

- user-service/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./ RUN
npm install --omit=dev
COPY server.js .
EXPOSE 3000
CMD ["npm", "start"] □
user-service/server.js
```

```
const express = require('express'); const
mongoose = require('mongoose'); const app =
express(); app.use(express.json()); const
PORT=3000,
MONGO_URL=process.env.MONGO_URL || 'mongodb://localhost:27017/bookflow_users'; const
User=mongoose.model('User',new
mongoose.Schema({userId:{type:Number,unique:true},name:String,role:String}));
```

```

app.get('/health',(req,res)=>res.json({service:'bookflow-
userservice',week:'10',author:'Kamtam Priyanka',status:'UP'}));
app.get('/users',async(req,res)=>res.json(await User.find().select('-_id -
__v')));
app.post('/users',async(req,res)=>{const u=await
User.create(req.body);res.status(201).json({userId:u.userId,name:u.name,role:u
.role});});
async function start(){try{await mongoose.connect(MONGO_URL);const e=await
User.findOne({userId:501});if(!e)await User.create({userId:501,name:'Kamtam
Priyanka',role:'Student'});app.listen(PORT,'0.0.0.0',()=>console.log('User Service
running on port
3000'));}catch(e){console.error(e.message);process.exit(1)}} start();

```

- user-service/package.json

```

{"name":"bookflow-
userservice","version":"1.0.0","main":"server.js","scripts":{"start":"node
server.js"},"dependencies":{"express":"4.21.2","mongoose":"8.9.5"}}

```

- stock-service/Dockerfile

```

FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build COPY
pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /build/target/bookflow-stock-service-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]

```

- stock-service/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?><project
xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-
4.0.0.xsd"><modelVersion>4.0.0</modelVersion><parent><groupId>org.springframework
ork.boot</groupId><artifactId>spring-boot-
starterparent</artifactId><version>3.5.5</version><relativePath/></parent><groupId>c
o
m.bookflow.week10</groupId><artifactId>bookflow-
stockservice</artifactId><version>1.0.0</version><properties><java.version>17</java
.version></properties><dependencies><dependency><groupId>org.springframework.b
oot</groupId><artifactId>spring-boot-
starterweb</artifactId></dependency><dependency><groupId>org.springframework.boot</g

```

```

<groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
datajpa</artifactId></dependency><dependency><groupId>org.postgresql</groupId><art
ifactId>postgresql</artifactId><scope>runtime</scope></dependency></dependenci
es><build><plugins><plugin><groupId>org.springframework.boot</groupId><artifac
tId>spring-boot-maven-plugin</artifactId></plugin></plugins></build></project>

```

- stockservice/src/main/java/com/bookflow/week10/StockServiceApplication.java

```

package com.bookflow.week10; import
org.springframework.boot.SpringApplication; import
org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication public class StockServiceApplication { public
static void main(String[]
args){SpringApplication.run(StockServiceApplication.class,args);} }

```

- stockservice/src/main/java/com/bookflow/week10/controller/StockController.java

```

package com.bookflow.week10.controller; import com.bookflow.week10.entity.*;
import com.bookflow.week10.repository.*; import
org.springframework.http.ResponseEntity; import
org.springframework.web.bind.annotation.*; import java.util.Map;
@RestController public class StockController { private final
InventoryRepository inventory; private final LibraryOrderRepository orders; public
StockController(InventoryRepository i,LibraryOrderRepository
o){inventory=i;orders=o;}
@GetMapping("/health") public Map<String,String> health(){return
Map.of("service","bookflow-stock-service","week","10","author","Kamtam
Priyanka","status","UP");}
@GetMapping("/inventory/{bookId}") public ResponseEntity<?> get(@PathVariable Long
bookId){return
inventory.findById(bookId).map(ResponseEntity::ok).orElse(ResponseEntity.notFo
und().build());}
@PostMapping("/orders") public ResponseEntity<?> order(@RequestBody
Map<String,Long> req){Long u=req.get("userId"),b=req.get("bookId"); Inventory
i=inventory.findById(b).orElse(null); if(i==null)return
ResponseEntity.notFound().build(); if(i.getAvailableStock()<=0)return
ResponseEntity.status(409).body(Map.of("status",409,"error","OUT_OF_STOCK","me
ssage","Book "+b+" is out of stock"));
i.setAvailableStock(i.getAvailableStock()-1);inventory.save(i);LibraryOrder
o=orders.save(new LibraryOrder(u,b,"BORROWED"));return
ResponseEntity.status(201).body(Map.of("id",o.getId(),"userId",o.getUserId(),"
bookId",o.getBookId(),"status",o.getStatus()));}

```

- stock-service/src/main/java/com/bookflow/week10/entity/Inventory.java

```

package com.bookflow.week10.entity; import jakarta.persistence.*; @Entity
@Table(name="inventory") public class Inventory { @Id private Long bookId;
private Integer totalStock; private Integer availableStock; public

```

```
Inventory(){ } public Inventory(Long id,Integer total,Integer
available){bookId=id;totalStock=total;availableStock=available;} public Long
getBookId(){return bookId;} public void setBookId(Long v){bookId=v;} public
Integer getTotalStock(){return totalStock;} public void setTotalStock(Integer
v){totalStock=v;} public Integer getAvailableStock(){return availableStock;}
public void setAvailableStock(Integer v){availableStock=v;} }
```

- stock-service/src/main/java/com/bookflow/week10/entity/LibraryOrder.java

```
package com.bookflow.week10.entity;
import jakarta.persistence.*;
@Entity @Table(name="orders") public
class LibraryOrder { @Id
@GeneratedValue(strategy=GenerationType.IDENTITY) private Long id; private
Long userId; private Long bookId; private String status; public
LibraryOrder(){ } public LibraryOrder(Long u,Long b,String
s){userId=u;bookId=b;status=s;} public Long getId(){return id;} public Long
getUserId(){return userId;} public Long getBookId(){return bookId;} public
String getStatus(){return status;} }
```

- stock-service/src/main/java/com/bookflow/week10/repository/InventoryRepository.java

```
package com.bookflow.week10.repository; import
com.bookflow.week10.entity.Inventory; import
org.springframework.data.jpa.repository.JpaRepository;
public interface InventoryRepository extends JpaRepository<Inventory,Long>{ }
```

- stockservice/src/main/java/com/bookflow/week10/repository/LibraryOrderRepository.java

```
package com.bookflow.week10.repository; import
com.bookflow.week10.entity.LibraryOrder; import
org.springframework.data.jpa.repository.JpaRepository;
public interface LibraryOrderRepository extends
JpaRepository<LibraryOrder,Long>{ }
```

- stock-service/src/main/resources/application.yml

```
spring:
datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/bookflow}
username: ${DB_USERNAME:postgres} password:
${DB_PASSWORD:postgres} jpa: hibernate: ddl-
auto: update show-sql: true server: port: 8080
```

- docker-compose.yml

```

services:
  gateway:
    build: ./gateway      image:
    bookflow-gateway:week10
    container_name: bookflow-gateway
    ports: ["8000:8000"]
    environment:
      USER_SERVICE_URL: http://user-service:3000
      STOCK_SERVICE_URL: http://stock-service:8080
    depends_on: [user-service, stock-service]    user-service:
      build: ./user-service      image:
      bookflow-user-service:week10
      container_name: bookflow-user-service
      ports: ["3000:3000"]      environment:
        MONGO_URL: mongodb://db-mongo:27017/bookflow_users
    depends_on: [db-mongo]    stock-service:
      build: ./stock-service      image:
      bookflow-stock-service:week10
      container_name: bookflow-stock-service
      ports: ["8080:8080"]      environment:
        DB_URL: jdbc:postgresql://db-postgres:5432/bookflow
        DB_USERNAME: postgres
      DB_PASSWORD: postgres      depends_on:
      [db-postgres]    db-postgres:
        image: postgres:16
      container_name: bookflow-postgres
      environment:      POSTGRES_DB:
      bookflow
        POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres      ports: ["5432:5432"]
      volumes: [postgres-data:/var/lib/postgresql/data]
    db-mongo:
      image: mongo:7
      container_name: bookflow-mongo
      ports: ["27017:27017"]
      volumes: [mongo-data:/data/db]
      volumes:
        postgres-data:
        mongo-data:

```

❏ k8s/deployment.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: bookflow-gateway
labels:

```

```

    app: bookflow-gateway
owner: kamtam-priyanka spec:
  replicas: 2
selector:
matchLabels:
  app: bookflow-gateway
template:
  metadata:
labels:
  app: bookflow-gateway
owner: kamtam-priyanka spec:
  containers:
    - name:
gateway      image: bookflow-
gateway:week10
imagePullPolicy: IfNotPresent
ports:
  - containerPort: 8000

```

❏ k8s/service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: bookflow-gateway-service
labels:
  app: bookflow-gateway
owner: kamtam-priyanka spec:
  type: LoadBalancer
selector:
  app: bookflow-gateway
ports:
  - protocol: TCP
port: 8000
targetPort: 8000

```

My practical task on docker application:

 **docker desktop**

PERSONAL

Q Search

Ctrl+K









K







 Gordon

 Containers

 Images

 Logs

 Volumes

 Kubernetes

 Builds

 Models

 MCP Toolkit BETA

 Docker Hub

 Extensions

Containers [Give feedback](#)

Container CPU usage ⓘ
0.86% / 800% (8 CPUs available)

Container memory usage ⓘ
481.15MB / 7.51GB

[Show charts](#)

Q Search

 ☐ Only show running containers

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
 bookflow-week -		-	-	1.86%	4 hours ago	   

Showing 1 item

Engine running

RAM 5.96 GB CPU 5.27% Disk: 8.77 GB used (limit 1006.85 GB)

 [Update version](#)